

Podczas kolokwium wszystkie działania wykonywane na ekranie komputera są rejestrowane i mogą zostać odtworzone przez prowadzących. Po napisaniu kolokwium, katalog z projektem należy spakować do formatu zip lub tar.gz i wysłać go przy użyciu kampusu. Proszę nie wyłączać komputera. Podczas kolokwium można korzystać ze wszystkich zasobów internetu, w tym z samodzielnie przygotowanych materiałów z wyłączeniem stron, aplikacji i pluginów opartych o sztuczną inteligencję. Podczas kolokwium zabroniona jest wzajemna komunikacja oraz publikowanie i dostęp do kodu stworzonego na potrzeby kolokwium. Zabronione jest korzystanie z innych urządzeń niż udostępniony komputer. Przedstawione poniżej kroki stanowią propozycję kolejności rozwiązywania zadania. Po przeczytaniu całości można zdecydować o rozwiązywaniu w innej kolejności. Do zadanych klas można dodawać pola i metody według własnego uznania, jeżeli pomoże to w rozwiązaniu zadania. W poleceniach, gdzie sposób implementacji nie jest podany można zastosować dowolne podejście. Tam, gdzie jest ono zaproponowane, inny sposób implementacji też jest dozwolony, ale nie będzie maksymalnie punktowany. Zadanie polega na napisaniu serwera sieciowego do gry w papier, kamień, nożyce. Po zalogowaniu i autentykacji, gracze mogą wyzywać się nawzajem na pojedynki. Każda gra to pojedyncza wymiana gestów, która może zakończyć się zwycięstwem jednego z graczy lub remisem. Wyniki każdego pojedynku są prezentowane jego uczestnikom po jego zakończeniu, a lista rankingowa jest przechowywana w bazie danych.

Etap 1. Serwer z autentykacją

Krok 1. ★☆☆

Utwórz pakiet server, a w nim napisz klasę Server. W klasie powinna być metoda listen(), która w nieskończonej pętli oczekuje na połączenia od klientów. W klasie Server utwórz metodę main(), która uruchomi aplikację.

Krok 2. ★★☆☆

W pakiecie server utwórz klasę ClientHandler służącą do obsługi pojedynczego klienta przez serwer. Niech obiekt tej klasy będzie tworzony przez serwer w momencie dołączenia klienta. Serwer powinien obsługiwać połączenia od wielu klientów jednocześnie. Obiekt klasy Server powinien mieć możliwość wysyłania wiadomości klientom.

Krok 3. ★★☆☆

W obiekcie klasy Server przechowuj listę obiektów ClientHandler obsługujących aktualnie połączonych klientów. Obsłuż dodanie klienta do tej listy w momencie połączenia i usunięcie go w momencie odłączenia od serwera.

Krok 4. ★☆☆

W pakiecie server umieść udostępnioną klasę Database. Znajduje się w niej metoda authenticate() zwracająca zawsze prawdę. Otrzyma ona bardziej złożoną implementację w kolejnych krokach. Umieść obiekt tej klasy w klasie Server i pozwól obiektom ClientHandler na dostęp do jej metody authenticate().

Krok 5. ★★☆☆

Niech po połączeniu, serwer wyśle klientowi pytanie o login, a następnie o hasło. Otrzymane

dane niech wykorzysta w metodzie `authenticate()` z klasy `Database`. Jeżeli zwróci fałsz, klient powinien zostać usunięty z serwera za pomocą metod stworzonych w kroku 3. W przypadku poprawnej autentykacji zapisz login w obiekcie klienta, aby w przyszłości mógł z niego skorzystać serwer.

Etap 2. Gra

Krok 1. ★★☆☆

Utwórz pakiet `game`. W nim utwórz typ wyliczeniowy `Gesture` o trzech wartościach - `ROCK`, `PAPER`, `SCISSORS`. Napisz w nim statyczną metodę `fromString()`, która przekształci napisy na obiekty tego typu: "r" na `ROCK`, "p" na `PAPER`, a "s" na `SCISSORS`.

Krok 2. ★★☆☆

W typie `Gesture` utwórz metodę `compareTo()`, która przyjmie inny obiekt `Gesture`. Jeżeli są to takie same gesty, należy zwrócić 0. Gdy metoda jest wywoływana na rzecz wygrywającego gestu, a w argumencie otrzymuje przegrywającego powinna zwrócić 1. W przeciwnym przypadku niech zwróci -1 (`ROCK < PAPER`, `PAPER < SCISSORS`, `SCISSORS < ROCK`).

Krok 3. ★☆☆☆

W pakiecie `game` utwórz (póki co pustą) klasę `Duel` reprezentującą pojedynczą grę i klasę `Player` reprezentującą każdego z jej uczestników. W klasie `Player` stwórz pola i metody zgodnie ze schematem, ale jeszcze ich nie implementuj. Symbole widoczności metod zostały zastąpione znakiem zapytania. Na późniejszym etapie samodzielnie dobrać widoczność tak, aby była maksymalnie restrykcyjna, ale pozwalała na działanie aplikacji. Do klasy `Player` nie można dopisać dodatkowych pól i metod.

Player
-duel : Duel
? makeGesture(gesture : Gesture) : void
? enterDuel(duel : Duel) : void
? leaveDuel() : void
? isDuelling() : boolean

Krok 4. ★☆☆☆

W klasie `Duel` przechowuj informację o dwóch uczestnikach (`Player`) i ich gestach (`Gesture`). Uczestników należy przyjąć w konstruktorze i wywołać ich metody `enterDuel()` tak, aby ustawić graczom w ich obiektach aktualny pojedynek. Ich metoda `leaveDuel()`, powinna ustawiać w polu `duel` wartość `null`, a metoda `isDuelling()` sprawdzać, czy wartość w polu `duel` jest różna od `null`.

Krok 5. ★☆☆☆

Przy użyciu `JUnit 5` napisz test jednostkowy, który tworzy graczy, pojedynek z tymi graczami i sprawdza, czy dla obu graczy metoda `isDuelling()` zwróci prawdę.

Krok 6. ★★☆☆

W klasie `Duel` napisz metodę `public void handleGesture(Player player, Gesture gesture)`. Metoda powinna przyjąć gest od gracza i go zapisać. Metoda `makeGesture()` w klasie `Player` powinna wywołać metodę `handleGesture()` aktualnego pojedynku.

Krok 7. ★★☆☆

W klasie `Duel` napisz zagnieżdżony rekord `Result` zawierający dwa pola typu `Player`: `winner` i `loser`. W klasie `Duel` napisz publiczną, bezargumentową metodę `evaluate()`, która w przypadku zwycięstwa jednego z graczy zwróci obiekt `Result`, a w przypadku remisu - `null`.

Krok 8. ★☆☆

Napisz dwa testy jednostkowe metody `evaluate()`: jeden sprawdzający wygraną jednego z graczy, drugi sprawdzający remis.

Krok 9. ★☆☆

Znajdź odpowiednie miejsce do wywołania metody `leaveDuel()`.

Etap 3. Integracja gry z serwerem

Krok 1. ★☆☆

W klasie `Server` napisz dwie metody, które zostaną zaimplementowane w kolejnych krokach:

- `public void challengeToDuel(ClientHandler challenger, String challengeeLogin)`,
- `private void startDuel(ClientHandler challenger, ClientHandler challengee)`.

Niech klasa `ClientHandler` dziedziczy po `Player`. Jeżeli klient wyśle wiadomość, należy wywołać metodę `challengeToDuel()`, przekazując tę wiadomość jako `challengeeLogin`.

Krok 2. ★★☆☆

Zaimplementuj metodę `challengeToDuel()` tak, by serwer przeszukał listę klientów i jeżeli znajduje się w niej klient o tym loginie, niech wywoła metodę `startDuel()`, przekazując jej obu uczestników pojedynku. Jeżeli nie ma osoby o tym loginie, należy napisać to klientowi rzucającemu wyzwanie.

Krok 3. ★☆☆

Zaimplementuj metodę `startDuel()` tak, by tworzyła obiekt `Duel` i wysyłała obu klientom informację o rozpoczęciu pojedynku.

Krok 4. ★★☆☆

Jeżeli klient wyśle wiadomość podczas trwania pojedynku, należy tę wiadomość przekształcić na `Gesture` i wywołać odziedziczoną metodę `makeGesture()`. Dopuszczalne są wyłącznie wiadomości "p", "r", "s" - odpowiadające gestom w grze. Pozostałe wiadomości powinny zostać zignorowane.

Krok 5. ★★★★★

W klasie `Duel` stwórz prywatne pole `onEnd` będące referencją na obiekt interfejsu funkcyjnego, którego metoda `nic` nie przyjmuje i nic nie zwraca. Interfejs stwórz sam lub wybierz odpowiedni z istniejących. Napisz mutator do tego pola. Metoda tego interfejsu

powinna wywołać się, gdy w pojedynku zostaną zarejestrowane gesty obu graczy. Zwróć uwagę, żeby nie uszkodzić napisanych wcześniej testów jednostkowych.

Krok 6. ★★☆☆

W metodzie startDuel() klasy Server wywołaj mutator pola onEnd przekazując mu funkcję, która wywołuje metodę evaluate() pojedynku i w zależności od jej wyniku informuje graczy o remisie, zwycięstwie lub porażce.

Krok 7. ★☆☆☆

Zabroń wyzwania na pojedynek samego siebie oraz gracza, który aktualnie pojedynkuje się z kimś innym. W takim przypadku należy wysłać wyzywającemu stosowną wiadomość.

Etap 4. Zapis wyników

Przygotowana została baza danych w postaci pliku SQLite, która zawiera tabelę "users" postaci:

login	password	points
alice	s	0
bob	e	0
charlie	c	0
dave	r	0
eve	e	0
frank	t	0

Krok 1. ★★☆☆

Zaimplementuj metodę authenticate(), aby sprawdzała login i hasło w bazie.

Krok 2. ★★☆☆

Zaimplementuj metodę updateLeaderboard() tak, aby dodawała punkt zwycięzcy i zabierała go przegranemu. Wywołaj tę metodę po każdej nieremisowej grze.

Krok 3. ★★★

Zaimplementuj metodę getLeaderboard(), by zwracała mapę, której kluczem jest login, a wartością liczba punktów. Niech lista rankingowa wyświetla się na konsoli serwera po każdej zakończonej grze, za pośrednictwem obiektu Server. Lista powinna zawierać loginy i wyniki graczy uszeregowane od największego do najmniejszego.

Po wysłaniu rozwiązań proszę nie wyłączać komputera.